

## ReactJS 解構 Props 和 States

在本章中，我們將了解在 **React** 中解構 **props** 和 **state**。解構是 **ES6** 的一項功能，可以將數組中的值或對像中的屬性解壓縮到不同的變量中。在 **React** 中，解構 **props** 和狀態提高了代碼的可讀性。

什麼是解構(destructuring)？

解構(destructuring)是在 **ES6** 中引入的。這是一個 **JavaScript** 特性，允許我們從陣列或物件中提取數據並將它們分配給它們自己的變數。

假設我們有一個具有以下屬性的員工物件：

```
const employee = {  
  firstName: "Amy",  
  lastName: "Lin",  
  emailId: "amy@gmail.com"  
};
```

在 **ES6** 之前，你必須單獨訪問每個屬性：

```
console.log(employee.firstName) // Amy  
console.log(employee.lastName) // Lin  
console.log(employee.emailId) // amy@gmail.com
```

解構讓我們簡化這段代碼：

```
const { firstName, lastName, emailId } = employee ;
```

相當於

```
const firstName = employee.firstName  
const lastName = employee.lastName  
const emailId = employee.emailId
```

所以現在我們可以在沒有員工的情況下訪問這些屬性。字首：

```
console.log(employee.firstName) // Amy  
console.log(employee.lastName) // Lin  
console.log(employee.emailId) // amy@gmail.com
```

功能組件中的解構道具

員工 **function** 元件

讓我們使用以下程式碼建立一個名為 **Employee** 的功能元件：

```
import React from 'react'
export const Employee = (props) => {
  return (
    <div>
      <h1> Employee Details</h1>
      <h2> First Name : {props.firstName} </h2>
      <h2> Last Name : {props.lastName} </h2>
      <h2> EmailId : {props.emailId} </h2>
    </div>
  )
}
```

請注意，我們使用 **props** 訪問 **Employee** 元件中的員工數據。

應用組件

我們來看看 **App** 組件。我們使用 **props** 將員工 **firstName**、**lastName** 和 **email** 傳遞給 **Employee** 組件：

```
import React from 'react';
import logo from './logo.svg';
import './App.css';
import { Employee } from './components/Employee';
function App() {
  return (
    <div className="App">
      <header className="App-header">
        <Employee firstName = "Amy" lastName = "Lin" emailId =
"amy@gmail.com" />
      </header>
    </div>
  );
}
export default App;
```

**function** 元件中 **props** 的兩種解構方式

有兩種方法可以在功能元件中解構 props

第一種方法是在函數參數本身中解構它：

```
import React from 'react'
export const Employee = ({firstName, lastName, emailId}) => {
  return (
    <div>
      <h1> Employee Details</h1>
      <h2> First Name : {firstName} </h2>
      <h2> Last Name : {lastName} </h2>
      <h2> Eamil Id : {emailId} </h2>
    </div>
  )
}
```

第二種方式是在函數體中解構 props：

```
import React from 'react'
export const Employee = props => {
  const {firstName, lastName, emailId} = props;
  return (
    <div>
      <h1> Employee Details</h1>
      <h2> First Name : {firstName} </h2>
      <h2> Last Name : {lastName} </h2>
      <h2> Eamil Id : {emailId} </h2>
    </div>
  )
}
```

類別元件中的解構 props

讓我們先看看不解構我們的程式碼是什麼樣子：

```
import React, { Component } from 'react'
class Employee extends Component {
  render() {
    return (
      <div>
        <h1> Employee Details</h1>
        <h2> First Name : {this.props.firstName} </h2>
        <h2> Last Name : {this.props.lastName} </h2>
        <h2> Eamil Id : {this.props.emailId} </h2>
      </div>
    )
  }
}
export default Employee;
```

現在，讓我們解構我們的 props，

這樣我們就可以擺脫每個 props 前面的 this. Props。

在類別元件中，我們在 render()函數中解構 props：

```
import React, { Component } from 'react'
class Employee extends Component {
  render() {
    const {firstName, lastName, emailId} = this.props;
    return (
      <div>
        <h1> Employee Details</h1>
        <h2> First Name : {firstName} </h2>
        <h2> Last Name : {lastName} </h2>
        <h2> Eamil Id : {emailId} </h2>
      </div>
    )
  }
}
export default Employee;
```

## 解構 state

解構狀態類似於道具。下面是 **React** 中解構狀態的語法：

```
import React, { Component } from 'react'
class StateDemo extends Component {
  render() {

    const {state1, state2, state3} = this.state;

    return (
      <div>
        <h1> State Details</h1>
        <h2> state1 : {state1} </h2>
        <h2> state2 : {state2} </h2>
        <h2> state3 : {state3} </h2>
      </div>
    )
  }
}

export default StateDemo;
```

當我們從父元件傳遞 **props** 到子元件時，可以使用解構賦值方式簡化子元件內部對於 **props** 的存取。以下是一個使用解構賦值來訪問 **props** 的 **React** 元件的例子：

```
import React from 'react';
function UserCard({ name, email, phone }) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>{email}</p>
      <p>{phone}</p>
    </div>
  );
}

function App() {
```

```

const user = {
  name: 'John Doe',
  email: 'johndoe@example.com',
  phone: '123-456-7890'
};

return (
  <div>
    <UserCard {...user} />
  </div>
);
}

```

在這個例子中，我們建了一個名為 `UserCard` 的函數元件，並使用解構賦值方式將 `name`、`email` 和 `phone` 從 `props` 中取出，並使用它們來渲染 `UserCard` 的內容。

在 `App` 元件中，我們建了一個 `user` 物件，包含了使用者的名字、電子郵件地址和電話號碼。我們將這個 `user` 物件通過展開運算符 `{...user}` 傳遞給 `UserCard` 元件。這樣做可以將 `user` 物件中的所有屬性都傳遞給 `UserCard` 元件，相當於將 `name`、`email` 和 `phone` 三個屬性分別作為 `props` 傳遞給 `UserCard`。

在 `UserCard` 元件中，我們使用解構賦值方式將 `name`、`email` 和 `phone` 三個屬性從 `props` 中取出，並使用它們來渲染 `UserCard` 元件的內容。這樣做可以簡化對 `props` 的存取，使程式碼更加簡潔易讀。

### state 解構賦值

當我們在 `React` 元件中使用多個狀態時，使用解構賦值方式來訪問這些狀態可以更加方便。以下是一個使用多個狀態和解構賦值方式訪問它們的 `React` 元件的例子：

```

import React, { useState } from 'react';

function Form() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

```

```

function handleNameChange(event) {
  setName(event.target.value);
}

function handleEmailChange(event) {
  setEmail(event.target.value);
}

function handlePasswordChange(event) {
  setPassword(event.target.value);
}

function handleSubmit(event) {
  event.preventDefault();
  console.log(`Name: ${name}, Email: ${email}, Password: ${password}`);
}

return (
  <form onSubmit={handleSubmit}>
    <label>
      Name:
      <input type="text" value={name} onChange={handleNameChange} />
    </label>
    <label>
      Email:
      <input type="email" value={email} onChange={handleEmailChange} />
    </label>
    <label>
      Password:
      <input type="password" value={password}
onChange={handlePasswordChange} />
    </label>
    <button type="submit">Submit</button>
  </form>
);
}

```

在這個例子中，我們創建了一個名為 **Form** 的函數元件，並使用 **useState Hook** 來建立三個狀態變數：**name**、**email** 和 **password**。我們使用解構賦值方式將這

些狀態變數從 `useState` 的返回值中取出，並分別將它們命名為 `name`、`email` 和 `password`。

在這個例子中，我們使用了三個不同的事件處理函數：`handleNameChange`、`handleEmailChange` 和 `handlePasswordChange`。當這些事件被觸發時，它們會更新對應的狀態變數，從而實時反映在表單元素中。

最後，在表單的 `submit` 事件中，我們防止預設行為（即頁面刷新），並將表單中的三個值打印到控制台中。這樣使用解構賦值方式來訪問狀態，可以讓我們更輕鬆地訪問和更新多個狀態，使程式碼更加簡潔易讀。